

EE-2003

Computer Organization and Assembly Language

Week 02 – Lecture 01

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

August 26, 2025

Outline

- 1 Quick Review: What is Done?
- 2 Computer Organization & Computer Architecture
- 3 Why Assembly Language?
- 4 Internal Organization of Computer
- 5 CPU and Its Relationship with RAM/ROM
 - RAM Memory and its Addresses
 - Two Issues with RAM
- 6 Registers and Cache
- 7 CPU Main Objective: Fetch–Decode–Execute Cycle
- 8 Simple Assembly Example
- 9 Intel Software Developer Manual

Quick Review: What is Done?

- Course outline was discussed in detail
- History and evolution of microprocessors across different generations
- Evolution of **x86** and **x88** microprocessors

Quick Review: What is Done?

- Latest 64-bit architectures
- Concept of **Multicores**
- **Hyper-Threading (HT)** and **Simultaneous Multithreading (SMT)**
- Role of **Floating Point Units (FPUs)**

Quick Review: What is Done?

- Memory and its organization
- Capacity calculation and representation
- Examples of memory capacity:
 - **16K, 32K, 64K, ...**
- Examples of memory organization:
 - **$2K \times 8$, $16K \times 8$, ...**

Quick Review: Internal Organization of Computer

- CPU as the central component
- Connected to **Memory** and **I/O Devices**
- Communication through system buses:
 - **Data Bus** – transfers actual data
 - **Address Bus** – specifies memory or I/O location
 - **Control Bus** – manages read/write operations
- *This topic will be continued today in detail.*

Computer Organization & Architecture

- A computer has two important aspects:
 - **Architecture** – What the computer *looks like* to a programmer.
 - **Organization** – How the computer *is built* internally.
- In simple words:
 - Architecture = Design (What it should do?)
 - Organization = Implementation (How it is done?)

Computer Architecture (Easy Explanation)

- Deals with the **abstract design** of a computer.
- Includes:
 - Instruction Set (e.g., ADD, SUB, MOV)
 - Data types (8-bit, 16-bit, 32-bit, 64-bit)
 - Memory addressing modes
- Example:
 - A programmer only cares about **instructions and registers**, not the physical wiring.

Computer Organization (Easy Explanation)

- Deals with the **physical implementation**.
- Includes:
 - Control signals
 - How data moves inside the CPU
 - Memory hierarchy (Cache, RAM, ROM)
 - How buses connect CPU, memory, I/O
- Example:
 - Two computers may have the same architecture but different organizations (one faster, one slower).

Architecture vs Organization

Computer Architecture	Computer Organization
What the computer <i>does</i>	How the computer <i>works</i>
Visible to the programmer	Hidden from the programmer
Includes instruction set, data types, addressing modes	Includes control signals, memory hierarchy, bus systems
Concerned with design	Concerned with implementation
Example: x86, ARM architecture	Example: Intel i5 vs i7 processors (different organizations of same architecture)

Why Assembly Language?

- **Close to the Hardware** – Almost a readable form of Machine Language (0s and 1s).
- **Efficient** – Programs can run faster and use less memory compared to high-level languages.
- **Direct Hardware Control** – Can access CPU registers, memory locations, and I/O ports directly.
- **Learning Benefit** – Helps understand how computers actually work inside (CPU, memory, buses).
- **Foundation** – Strong base for system programming, embedded systems, and optimization.
- **Applications:**
 - Performance-critical parts of **Game Development** (graphics, console programming).
 - Low-level routines in **Machine Learning / AI accelerators** (GPU optimizations, SIMD instructions).

Example: Machine Code 10110000 vs. Assembly `MOV AX, BX`

Internal Organization of Computer

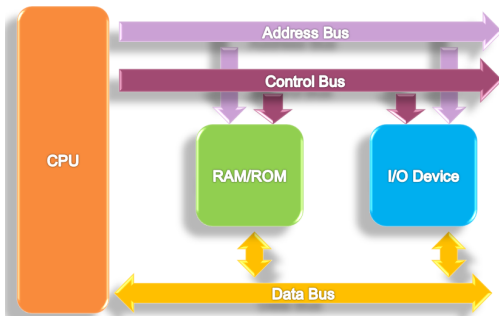


Figure: CPU connected with RAM and ROM through Address, Data, and Control Buses

- **CPU** communicates with memory through three types of buses:
 - **Address Bus** – selects memory location
 - **Data Bus** – transfers actual data
 - **Control Bus** – manages read/write operations
- **RAM (Random Access Memory)** – temporary, volatile memory used for active programs and data.
- **ROM (Read Only Memory)** – permanent memory storing firmware or boot instructions.

CPU and Its Relationship with RAM/ROM

- For the CPU to process information, the data must be present in **RAM** or **ROM**.
- **ROM (Read Only Memory):**
 - Provides fixed and permanent information.
 - Examples: tables for character patterns, firmware, initialization/boot programs.
- **RAM (Random Access Memory):**
 - Stores temporary information that changes with time.
 - Examples: running Operating System, application programs, user data.
- If required information is **not in RAM/ROM**, the CPU requests it from **secondary storage** (e.g., hard drives, SSDs) and loads it into RAM before processing.

RAM Memory and Its Addresses

- RAM is organized as a large array of memory cells.
- Each cell has a unique **address**.
- Addresses are arranged in a sequence:
 - **High Address** at the top
 - **Low Address** at the bottom

Address	Stored Value
0x1004	19
0x1003	(part of int x)
0x1002	(part of int x)
0x1001	(part of int x)
0x1000	(part of int x)
...	...

Figure: Example: RAM storing an int $x = 19$

C++ Example:

Code

Two Issues with RAM

- **Speed:** RAM is relatively **slow** compared to CPU speed.
 - CPU often waits for data to be fetched from RAM.
 - This is why cache memory is used to improve performance.
- **Instruction Tracking:** CPU must keep track of instructions and data locations in RAM.
 - Uses the **Program Counter (PC)** to know the next instruction.
 - Extra control logic is required for synchronization.

Why Registers and Cache?

- **Registers:**

- Located inside the CPU (very close to the processor).
- Extremely fast compared to Cache and RAM.
- Used for temporary storage of instructions, addresses, and data during execution.
- Very **expensive** and limited in number.

- **Cache Memory:**

- Sits between CPU and RAM.
- Stores frequently used instructions/data to reduce CPU waiting time.
- Faster than RAM, slower than registers.

- **RAM vs Registers:**

- RAM is very large but extremely slow compared to CPU speed.
- Cannot be fully built with registers. Why?

CPU Main Objective: Fetch–Decode–Execute Cycle

- The CPU works in a simple cycle called the **Fetch–Decode–Execute Cycle**.
- **Steps:**
 - ① **Fetch** – Get the instruction from RAM (at the address given by the Program Counter).
 - ② **Decode** – The Control Unit interprets the instruction (what operation needs to be done?).
 - ③ **Execute** – The CPU carries out the instruction (using ALU, registers, memory).
- This cycle repeats continuously while the computer is running.

Simple Assembly Instructions

Example Code

```
MOV AX, 11      ; Move value 11 into register AX
INC AX          ; Increment AX by 1
ADD AX, 5        ; Add 5 to AX
```

- **Instruction:** The command itself (e.g., MOV, INC, ADD).
- **Operands:** The data or registers on which the instruction works.
 - In MOV AX, 11:
 - Instruction = MOV
 - Operands = AX, 11
- After execution of above code: AX = 17

Intel Software Developer Manual

- The **Intel Software Developer's Manual (SDM)** is the official reference for Intel processors.
- Covers instruction sets and system programming details for:
 - **8-bit, 16-bit, 32-bit, and 64-bit architectures**
- Essential for understanding:
 - Assembly instructions
 - CPU modes and registers
 - System-level programming
- **Useful References (check in SDM):**
 - Page 55
 - Page 57
 - Page 67
 - Page 76

Link: Intel® 64 and IA-32 Architectures Software Developer Manuals

Thanks!

Any Questions?

Remember: Once you understand the **brain of the CPU**, everything else in computer systems will become much easier to grasp.

This is your foundation for mastering Computer Organization and Assembly Language!